

Agentic Coding Mini Project

Building and Using a Differentiable Coupled Climate Model

Team size: 4–5 students **Duration:** 2 weeks

Scientific Goal

Use agentic coding to build a minimal differentiable version of a coupled atmosphere–ocean quasi-geostrophic model. Validate the differentiable model against the original **qgs** implementation, then use gradients to estimate uncertain physical parameters from synthetic observations.

The main scientific question is:

Can automatic differentiation recover atmosphere–ocean coupling parameters from synthetic observations, and how does chaotic error growth limit the useful optimization window?

Prerequisites and Ideal Team Profile

This project is well suited to ML-leaning students; it requires *no* fluid dynamics, GFD, or prior qgs knowledge, because the physics is pre-projected into a fixed tensor (students never touch a PDE). What it does require:

Every team member

- **Scientific-Python fluency:** comfortable with NumPy semantics, broadcasting, and indexing. Non-negotiable: the validation effort is “does my array match qgs element-for-element, in the right order.”
- **A working coding-agent account** (Claude Code or Codex), *active before day 1*. At least 3 of 4–5 members should have one, or the agentic-coding work bottlenecks on a single person.
- **Skepticism toward generated code.** The core lesson is that an agent will hand you a frozen tensor with zero parameter dependence that looks correct and could silently fails Task 5. You want people whose instinct is to write the finite-difference check.

Concentrated on 1–2 members (need not be universal)

- **JAX experience** Specifically `grad/jacfwd`, `lax.scan`, the x64 gotcha, and forward- vs. reverse-mode AD. One fluent person can own `src/` and unblock the rest; *zero* fluent members is a real risk on a two-week timeline.
- **Dynamical-systems sense** (chaos, Lyapunov time, why a loss surface roughens with window length)
- **Basic optimization intuition** (step-size and finite-difference δ tuning).

If you can guarantee one specialized skill across the group, make it **JAX**; if two, add a **dynamical-systems thinker**. That pairing covers both the “can we build it” and “what does it tell us” halves of the project.

Starting Resources

- qgs repository: <https://github.com/Climdyn/qgs>

- qgs coupled atmosphere–ocean example: [qgs_maoam.py](#)
- qgs documentation: <https://qgs.readthedocs.io/>
- Claude Code overview: <https://code.claude.com/docs/en/overview>
- Claude Code project instructions (CLAUDE.md): <https://code.claude.com/docs/en/memory>
- JAX documentation: <https://docs.jax.dev/>

Background

The qgs package provides a reduced-order climate model with a two-layer quasi-geostrophic atmosphere coupled to a shallow-water ocean component (the MAOOAM configuration). Write the model abstractly as

$$\frac{d\mathbf{x}}{dt} = \mathbf{f}(\mathbf{x}, \boldsymbol{\theta}),$$

where $\mathbf{x}$ is the coupled atmosphere–ocean state vector, $\boldsymbol{\theta}$ is a vector of physical parameters, and $\mathbf{f}$ is the nonlinear tendency operator.

How qgs actually computes the tendency (read this first)

This is the single most important structural fact about qgs, and it shapes the entire project. After the PDEs are projected onto a spectral basis (Galerkin projection), the tendency is *exactly* a sparse quadratic form in the state vector. Internally qgs augments the state with a leading constant, $\tilde{\mathbf{x}} = (1, \mathbf{x})$, to absorb the constant forcing terms (insolation, etc.), and evaluates

$$\frac{dx_i}{dt} = \sum_{j,k} \mathcal{T}_{ijk} \tilde{x}_j \tilde{x}_k,$$

where $\mathcal{T}$ is a fixed third-order sparse tensor assembled once from the basis-function inner products and the physical parameters $\boldsymbol{\theta}$. There are no other nonlinearities and no hidden state. In qgs the tendency function is literally:

```
# qgs/functions/tendencies.py (simplified)
def f(t, x):
    xx = np.concatenate([1.0], x) # augment with constant
    return sparse_mul3(coo, val, xx, xx)[1:] # dx_i/dt = sum_jk T[i,j,k] xx[j] xx[k]
```

Consequence 1 — the JAX tendency is short.

Reimplementing $\mathbf{f}$ in JAX is essentially a single tensor contraction (`jnp.einsum` or a sparse equivalent) once the tensor $\mathcal{T}$ is exported. The intellectual work of this project is *not* the translation; it is (a) validating it rigorously and (b) making the tensor depend correctly on $\boldsymbol{\theta}$ so gradients are meaningful.

For parameter estimation we need $\partial \mathbf{f} / \partial \mathbf{x}$ and $\partial \mathbf{f} / \partial \boldsymbol{\theta}$, and gradients of a scalar objective $\nabla_{\boldsymbol{\theta}} L$.

Consequence 2 — you cannot freeze the whole tensor and still differentiate w.r.t. $\boldsymbol{\theta}$.

The parameters $\boldsymbol{\theta}$ are baked *into* the numbers in $\mathcal{T}$ when the tensor is built. If you export one frozen tensor and treat it as a constant, then $\partial \mathbf{f} / \partial \boldsymbol{\theta} \equiv 0$ structurally, and Task 5 silently fails. You must express how $\mathcal{T}$ depends on the parameters you intend to recover. Section “Making $\mathcal{T}$ depend on $\boldsymbol{\theta}$ ” below tells you how.

Scope

Do not rewrite the full qgs package. Build a minimal JAX implementation of *one fixed* coupled atmosphere–ocean configuration based on the existing `qgs_maoam.py` example (use `dynamic_T=False`; the recommended truncation gives a state-vector dimension of order a few tens, e.g. ~ 36).

The recommended workflow is:

1. Configure the original qgs model.
2. Export the fixed model tensor(s) and metadata.
3. Reimplement the tendency calculation in JAX.
4. Implement a fixed-step RK4 integrator in JAX.
5. Validate the JAX model against qgs.
6. Use automatic differentiation for parameter estimation.

Agentic Coding Requirement

Use Claude Code or an equivalent coding agent to assist with the translation. The agent is a tool, not an authority. Every generated function must be tested.

Create a project-level `CLAUDE.md` before starting. Use the following as a starting point:

```
# Project Goal

Create a minimal JAX implementation of one frozen qgs MAOOAM
configuration that is differentiable with respect to one or two
selected physical parameters.

# Rules

1. Do not rewrite the full qgs package.
2. Export the fixed tendency tensor from the original qgs model.
3. Use JAX with 64-bit precision (jax.config.update("jax_enable_x64", True)).
4. Implement a fixed-step RK4 integrator (qgs default is classic RK4).
5. Write tests before adding optimization experiments.
6. Compare tendencies against the original qgs model.
7. Compare automatic-differentiation gradients against finite differences.
8. The tensor depends on the physical parameters. Do NOT freeze a single
   tensor when you need gradients w.r.t. parameters: capture the
   parameter dependence explicitly (see project instructions).
9. Keep scientific scripts separate from library code.
10. Do not silently change physical parameter values.
11. Record assumptions and design decisions in README.md.
```

Save major agent prompts and summarize which tasks required human correction.

Task 1: Run the Original qgs Model

Clone and install qgs:

```
git clone https://github.com/Climdyn/qgs.git
cd qgs
conda env create -f environment.yml
```

```
conda activate qgs
python qgs_maoam.py
```

Confirm that the original model produces a trajectory. Record: state-vector dimension; timestep; spin-up (transient) length; integration length; parameter values; runtime.

Reference integrator.

The default qgs integrator (`RungeKuttaIntegrator`) is the classic fixed-step RK4 scheme. If your JAX RK4 uses the same Δt and the same tensor, the two trajectories should agree to near machine precision until chaotic divergence sets in. This is your strongest validation signal — use it.

Task 2: Export One Frozen Configuration

Write an export script that stores everything needed to reproduce one fixed model configuration *and* to differentiate it. qgs builds the tensor for you; you do not need to re-derive the inner products by hand:

```
from qgs.functions.tendencies import create_tendencies
# returns the assembled QgsTensor object alongside f and Df
f, Df, agotensor = create_tendencies(model_parameters, return_qgtensor=True)
coords = agotensor.tensor.coords # (3, nnz) integer index array
values = agotensor.tensor.data # (nnz,) float64 entries
ndim = model_parameters.ndim
```

The exported file should include:

- the sparse tensor coordinates (`coords`) and values (`values`);
- the state-vector dimension `ndim`;
- the selected physical parameter values;
- the timestep `dt`;
- variable names / state-vector slices (`model_parameters.variables_range`, latex var names);
- a reference initial condition (a post-spin-up state);
- a short reference trajectory for validation.

Use a portable format such as `npz`, `json`, or `NetCDF`. (qgs also offers `agotensor.save_to_file`; either is fine, but a plain `npz` of `coords/values` is the most transparent.) The goal is to separate the scientific model configuration from the JAX implementation.

Task 3: Build the Minimal JAX Model

3.1 The tendency

Implement a JAX function $f_{\text{JAX}}(\mathbf{x}, \boldsymbol{\theta})$ that reproduces the qgs contraction. With the exported tensor this is short:

```
import jax, jax.numpy as jnp
jax.config.update("jax_enable_x64", True)

def f_jax(x, T): # T is the dense (or sparse) (n+1,n+1,n+1) tensor
    xx = jnp.concatenate([jnp.ones(1), x])
    dxx = jnp.einsum("ijk,j,k->i", T, xx, xx)
    return dxx[1:]
```

For the recommended truncation the augmented dimension is small enough that a dense tensor is fine; if memory is a concern, keep `coords/values` and use a segment-sum contraction instead. Either way, verify that `einsum` and the qgs ordering of state variables match exactly.

3.2 The RK4 step

Implement one classic RK4 step and wrap it in `jax.lax.scan` for the trajectory:

$$\begin{aligned} \mathbf{k}_1 &= \mathbf{f}(\mathbf{x}_n), & \mathbf{k}_2 &= \mathbf{f}\left(\mathbf{x}_n + \frac{\Delta t}{2} \mathbf{k}_1\right), \\ \mathbf{k}_3 &= \mathbf{f}\left(\mathbf{x}_n + \frac{\Delta t}{2} \mathbf{k}_2\right), & \mathbf{k}_4 &= \mathbf{f}(\mathbf{x}_n + \Delta t \mathbf{k}_3), \\ \mathbf{x}_{n+1} &= \mathbf{x}_n + \frac{\Delta t}{6} (\mathbf{k}_1 + 2\mathbf{k}_2 + 2\mathbf{k}_3 + \mathbf{k}_4). \end{aligned}$$

Use JAX automatic differentiation (`jax.grad`, `jax.jacfwd`) for gradients. Reverse-mode differentiation through a `scan` of a few thousand steps of this small system is inexpensive — no gradient checkpointing is required.

Making $\mathcal{T}$ depend on θ (do this before Task 5)

This short section is the technical heart of the project and resolves Consequence 2 above. To recover a physical parameter θ you need a tensor that is an explicit function $\mathcal{T}(\theta)$. Two practical options:

1. **Linear-decomposition method (recommended).** In the MAOOAM tensor, the parameters that control coupling and dissipation enter the tensor entries *linearly* (e.g. ocean–atmosphere coupling d , bottom friction r , atmospheric friction k_d). For such a parameter,

$$\mathcal{T}(\theta) = \mathcal{T}_0 + \theta \mathcal{T}_1.$$

Recover the two pieces by building the qgs tensor at two different values of the parameter and finite-differencing the *tensor itself*:

$$\mathcal{T}_1 = \frac{\mathcal{T}(\theta_b) - \mathcal{T}(\theta_a)}{\theta_b - \theta_a}, \quad \mathcal{T}_0 = \mathcal{T}(\theta_a) - \theta_a \mathcal{T}_1.$$

Export $\mathcal{T}_0$ and $\mathcal{T}_1$. Then $\mathbf{f}_{\text{JAX}}(\mathbf{x}, \theta)$ uses $\mathcal{T}(\theta) = \mathcal{T}_0 + \theta \mathcal{T}_1$ and is exactly differentiable in θ . *Always confirm linearity* by building the tensor at a third value and checking the relation holds to round-off; if it does not, the parameter is not linear (see option 2).

2. **Rebuild-and-fit method (for nonlinear parameters).** Some parameters (notably the thermal-exchange / radiative coefficients) appear in several tensor terms and may enter nonlinearly. Build qgs tensors on a small grid of θ values and fit a low-order polynomial per nonzero entry, or restrict the project to the linear parameters above. For a first project we recommend choosing parameters that satisfy option 1.

Validate this step explicitly.

Before trusting any $\nabla_{\theta} L$, check $\partial \mathbf{f} / \partial \theta$ from your $\mathcal{T}(\theta)$ construction against a finite-difference of the qgs tendency itself: rebuild the qgs model at $\theta \pm \delta$, evaluate $\mathbf{f}_{\text{qgs}}$, and compare. This catches the most common silent failure in the whole project.

Task 4: Validate the Translation

Validation is the core requirement. Validate each layer before moving on.

4.1 Tendency comparison

Sample states from the original qgs trajectory. Compare $\mathbf{f}_{\text{qgs}}(\mathbf{x})$ and $\mathbf{f}_{\text{JAX}}(\mathbf{x})$ and report the relative tendency error

$$\epsilon_f = \frac{\|\mathbf{f}_{\text{JAX}} - \mathbf{f}_{\text{qgs}}\|_2}{\|\mathbf{f}_{\text{qgs}}\|_2}.$$

With a correct tensor export this should be at the level of float64 round-off ($\sim 10^{-12}$ or smaller). Anything larger means an index-ordering or augmentation-constant bug — fix it here, not later.

4.2 Short-trajectory comparison

Initialize the original and JAX models from the same state and compute

$$\epsilon_x(t) = \frac{\|\mathbf{x}_{\text{JAX}}(t) - \mathbf{x}_{\text{qgs}}(t)\|_2}{\|\mathbf{x}_{\text{qgs}}(t)\|_2}.$$

Because both use fixed-step RK4 with the same Δt , the trajectories should track to near machine precision initially; the error then grows exponentially at the leading Lyapunov rate of the system. Plotting $\log \epsilon_x(t)$ gives a clean estimate of the predictability time and motivates Task 6.

4.3 Gradient comparison

Choose a scalar loss

$$L = \frac{1}{N} \sum_{n=1}^N \|\mathbf{x}_{\boldsymbol{\theta}}(t_n) - \mathbf{x}_{\text{target}}(t_n)\|_2^2.$$

Compare automatic differentiation against centered finite differences,

$$\frac{\partial L}{\partial \theta_i} \approx \frac{L(\theta_i + \delta) - L(\theta_i - \delta)}{2\delta},$$

and report

$$\epsilon_{\nabla L} = \frac{\|\nabla L_{\text{AD}} - \nabla L_{\text{FD}}\|_2}{\|\nabla L_{\text{FD}}\|_2}.$$

Choose δ by a small sweep: too large and truncation error dominates, too small and round-off dominates. Note that for long windows in a chaotic system the loss surface is rough, so this comparison is only clean for *short* windows — which is itself one of the project’s findings.

4.4 Climate-statistic comparison

Run longer integrations after spin-up and compare: state-vector means; variances; selected probability density functions; autocorrelation functions; and one atmosphere–ocean metric chosen by the team. These should remain similar even after individual trajectories have diverged.

Task 5: Parameter-Recovery Experiment

Generate a synthetic truth trajectory using known parameters $\boldsymbol{\theta}_{\text{true}}$. Choose **one or two** uncertain parameters that enter the tensor linearly (recommended: ocean–atmosphere coupling d , bottom friction r , or atmospheric friction k_d — these satisfy the linear-decomposition method above). Create a perturbed initial estimate $\boldsymbol{\theta}_0 \neq \boldsymbol{\theta}_{\text{true}}$.

Define synthetic observations

$$\mathbf{y}(t_n) = \mathbf{H} \mathbf{x}_{\text{true}}(t_n),$$

where $\mathbf{H}$ selects a subset of atmospheric or oceanic variables. Estimate the parameters by minimizing

$$L(\boldsymbol{\theta}) = \frac{1}{N} \sum_{n=1}^N \|\mathbf{H} \mathbf{x}_{\boldsymbol{\theta}}(t_n) - \mathbf{y}(t_n)\|_2^2.$$

Use gradient descent or a JAX-compatible optimizer (e.g. `optax.adam`, or `jax.scipy.optimize / jaxopt` for L-BFGS). Because you are optimizing one or two scalars, gradient descent with a tuned step size is usually sufficient.

Task 6: Effect of Window Length

Repeat the parameter-recovery experiment using several optimization windows $T \in \{T_1, T_2, T_3, T_4\}$ (a sequence of increasingly long but computationally manageable windows; relate them to the predictability time measured in Task 4.2). For each window length, report: initial loss; final loss; gradient norm $\|\nabla_{\boldsymbol{\theta}} L\|_2$; recovered parameters; and the parameter-recovery error

$$\epsilon_{\boldsymbol{\theta}} = \frac{\|\hat{\boldsymbol{\theta}} - \boldsymbol{\theta}_{\text{true}}\|_2}{\|\boldsymbol{\theta}_{\text{true}}\|_2}.$$

Expect a sweet spot: windows too short are under-informative; windows too long make the loss surface rough and the gradient unreliable.

Questions to Answer

Your final notebook and presentation should answer:

1. Can the JAX tendency reproduce the original qgs tendency (to what precision)?
2. How long do the original and JAX trajectories remain close, and what does that imply about predictability?
3. Do long-run climate statistics remain similar after trajectories separate?
4. Do automatic-differentiation gradients agree with finite differences, and over what window lengths?
5. Can one or two physical parameters be recovered from synthetic observations?
6. How does optimization-window length affect gradient quality and parameter recovery?
7. Which parts of the translation were accelerated by agentic coding?
8. Which parts required scientific judgment or manual correction — especially the parameter-dependence of the tensor?

Deliverables

1. A minimal JAX implementation.
2. A qgs configuration/tensor-export script.
3. Automated validation tests (tendency, gradient, parameter-dependence).
4. One reproducible analysis notebook.

5. A short README with setup and run instructions.
6. A project-level CLAUDE.md.
7. A short record of major agent prompts and human corrections.

Suggested Repository Structure

```
differentiable-qgs/
|-- README.md
|-- CLAUDE.md
|-- environment.yml
|-- export_qgs_config.py
|-- configs/
| ' -- maoam_frozen_config.npz # T0, T1, ndim, dt, ic, var names
|-- src/
| |-- qgs_jax.py # f_jax(x, theta) -> dx/dt
| ' -- rk4.py # rk4_step + scan-based trajectory
|-- tests/
| |-- test_tendencies.py # eps_f vs qgs
| |-- test_param_dependence.py # df/dtheta vs qgs rebuilt at theta +/- d
| ' -- test_gradients.py # AD vs finite differences
|-- notebooks/
| ' -- parameter_recovery.ipynb
' -- prompts/
    ' -- agent_log.md
```

Suggested Team Split

- Student 1: original qgs setup, reference runs, and tensor export (including $\mathcal{T}_0$, $\mathcal{T}_1$).
- Student 2: JAX tendency implementation and tensor contraction.
- Student 3: RK4 implementation, tests, and gradient validation.
- Student 4: parameter-recovery and window-length experiments.
- Student 5: climate-statistic evaluation, prompt log, figures, and notebook integration.

With four students, combine the third and fifth roles.

Note: the translation (Tasks 3–4.1) is deliberately short because the qgs tendency is a single tensor contraction. Most of your effort and most of the scientific payoff are in the parameter-dependence step and in Tasks 5–6.

Stretch Goals

Attempt these only after completing the required tasks.

- Compare a trajectory loss with a climate-statistic loss:

$$L_{\text{clim}} = w_{\mu} \|\boldsymbol{\mu}_{\boldsymbol{\theta}} - \boldsymbol{\mu}_{\text{true}}\|_2^2 + w_{\sigma} \|\boldsymbol{\sigma}_{\boldsymbol{\theta}} - \boldsymbol{\sigma}_{\text{true}}\|_2^2.$$

- Estimate parameters using only atmospheric observations.

- Estimate parameters using only ocean observations.
- Add observational noise to $\mathbf{y}$.
- Compare gradient descent with a derivative-free optimizer.
- Compute finite-time sensitivity / Lyapunov growth from $\partial\mathbf{x}(t)/\partial\mathbf{x}(0)$ via AD.
- Use gradients to identify which state variables are most informative for parameter estimation.
- Recover a nonlinear-in- θ parameter (e.g. a thermal-exchange coefficient) using the rebuild-and-fit method.

Interpretation Guidance

A successful project does not require perfect parameter recovery. Scientifically useful outcomes include:

- gradients become unreliable as the optimization window grows past the predictability time;
- atmospheric variables constrain some parameters better than ocean variables (and vice versa for slow coupling parameters);
- trajectory fitting improves short forecasts but not climate statistics;
- the coding agent accelerates the translation but introduces subtle scientific errors (e.g. a frozen tensor with no parameter dependence, or a state-variable ordering mismatch) that only finite-difference validation reveals.

The main lesson is:

A differentiable climate model is useful only when the translation is scientifically validated *and* the model's dependence on its physical parameters is represented correctly, so that the gradients remain informative for the chosen problem.

Final Advice

Keep the port minimal, it is mostly one tensor contraction. Spend your time on the parameter-dependence of the tensor and on validation. Validate each layer before adding the next. Treat every agent-generated change as a hypothesis that must be tested against the original qgs model.